



CarbonLens

"Turning energy data into climate insight."

Final Project Report

Authored By:

Karthik Kesavarapu, Shri Chatrati, Shounak Sawant,
Rahee Chauhan

GitHub: <https://github.com/k15karthik/CarbonLens>

Table of Contents

I	Introduction	<u>4-5</u>
1a	Project Description	4
1b	Scope of the Data	4
1c	Appliances Chosen	4-5
II	Cleaning Up the Data	<u>5-14</u>
2a	Defined Process of Cleaning Process	5
2b	Cleaned Scripts	6-14
III	Exploratory Data Analysis	<u>14-18</u>
3a	Refrigerators EDA	14-15
3b	Room Air Conditioners EDA	15
3c	Televisions EDA	16
3d	Ceiling Fans EDA	17
3e	Ovens EDA	18
IV	Visualizations	<u>19-30</u>
4a	Televisions Visualizations	19-21
4b	Ceiling Fans Visualizations	21-23
4c	Air Conditioner Visualization	23-24
4d	Refrigerator Visualizations	25-26
4e	Ovens Visualizations	27-28
4f	Multivariable Visualizations	28-30
V	Machine Learning Analysis	<u>30-44</u>
5a	Ovens Machine Learning	30-34
5b	Ceiling Fans Machine Learning	34-36

5c	Air Conditioner Machine Learning	36-38
5d	Television Machine Learning	38-40
5e	Refrigerators Machine Learning	40-44
VI	Conclusion	<u>44</u>
6a	Changes From Project Report	44
6b	Reflections	44
VII	References	44

I. Introduction

This section is authored by Karthik Kesavarapu

1a. Project Description

As there is an increasing concern about the prevalent issues that are starting to arise with global warming, consumers are starting to become more aware of the products that release more carbon emissions. However, some applications may not include specific carbon data, and if they do consumers are not sure if it is the appropriate and eco-friendly product which ultimately leaves buyers unaware of the true environmental cost of their choices. *CarbonLens* aims to solve this problem by predicting the CO₂ emissions of appliances using machine learning, based on publicly available features such as energy usage, materials, and origin.

The purpose of this application is to hypothesize whether or not a product's CO₂ emissions can be accurately estimated with quantitative and qualitative features being predicted. In other words, *CarbonLens* proposes that machine learning can estimate the carbon footprint of appliances based on standard specification data, which will increase customer awareness of environmental impact.

The project is implemented in Python and with various Python libraries such as Pandas, Matplotlib, Datetime, ScikitLearn, Seaborn, and Counter.

1b. Scope of the Data

CarbonLens aims to answer questions about appliances in the scope of the United States of America. That is, all of the appliances talked about in this document, further extending to the actual datasets used to conduct the analysis, will be appliances that have an origin in the United States of America. The datasets used that fit this exact scope is the Energy Star Certified Product Data Sets [1].

1c. Appliances Chosen

The methodology used to choose the various appliances was by using a **Simple Random Sample** (SRS). The sample that has been selected are from categories within the Energy Star Certified Product Data Sets that people are expected to

use almost daily which include appliances, heating and cooling, electronics, home office, and lighting and fans. From these categories, we have selected refrigerators, room air conditioners, televisions, ceiling fans, and ovens. This is a good stratification of 5 different types of appliances that will show the overall trend of carbon dioxide usage in daily appliances.

II. Cleaning Up the Data

This section is authored by Karthik Kesavarapu

2a. Defined Process of Cleaning Data

The cleaning of the data occurred in this ten-step process which was done entirely by using Pandas. Initially, the duplicates, empty columns, rows with critical null values, and invalid energy values were dropped. After deleting and scraping each dataset from its null values, the process converts values that are string into proper integer types. For features that are encoded with a simple “Yes” or a “No”, those columns were transformed into 1/0. Furthermore, the brand names became standardized as well. Finally, all of the dates that were in the datasets were converted into the correct *datetime* object. Afterwards, the process resets the index of the *dataframe*. This completed and cleaned *dataframe* was then saved into a file.

The process defined here aims to emulate the same process used in an architecture in data science known as *Medallion Architecture* which is a way to clean raw data into business-level data which can be used for streaming analytics, reporting, and machine learning analysis.

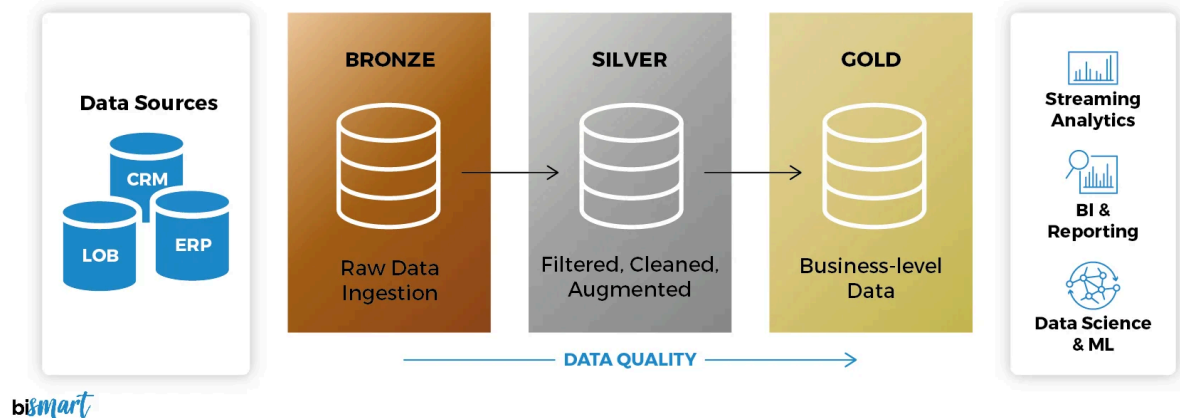


Figure 1: Medallion Architecture Pipeline

2b. Cleaned Scripts

The cleaned datasets can be viewed in the github repository for CarbonLens, and the original data is found on the website for the Energy Star Certified Product Data Sets. However, below will be the python scripts used to execute the cleaning pipeline as described in *Section 2a*.

```
dfRefrigerator = pd.read_csv('Residential Refrigerators.csv')
# Display the first few rows of the dataframe
print(dfRefrigerator.head())
print(dfRefrigerator.columns.tolist())

# 1. Drop duplicates
dfRefrigerator = dfRefrigerator.drop_duplicates()

# 2. Remove completely empty columns
dfRefrigerator = dfRefrigerator.dropna(axis=1, how='all')

# 3. Remove rows missing key values
dfRefrigerator = dfRefrigerator.dropna(subset=["Brand Name", "Model Number",
"Annual Energy Use (kWh/yr)", "Additional Model Information"])

# 4. Convert numeric columns to proper types
numeric_cols = [
    "Annual Energy Use (kWh/yr)",
    "US Federal Standard (kWh/yr)",
    "Percent Less Energy Use than US Federal Standard",
    "Height (in)",
    "Width (in)",
    "Capacity (Total Volume) (ft3)"
]
dfRefrigerator[numeric_cols] = dfRefrigerator[numeric_cols].apply(pd.to_numeric,
errors="coerce")

# 5. Drop rows with invalid or 0 energy values
dfRefrigerator = dfRefrigerator[dfRefrigerator["Annual Energy Use (kWh/yr)"] > 0]
```

```

# 6. Standardize brand names
dfRefrigerator["Brand Name"] = dfRefrigerator["Brand Name"].str.strip().str.title()

# 7. Convert all Yes/No columns to 1/0
for col in dfRefrigerator.columns:
    if dfRefrigerator[col].isin(['Yes', 'No']).any():
        dfRefrigerator[col] = dfRefrigerator[col].replace({'Yes': 1, 'No': 0})

# 8. Reset index
dfRefrigerator = dfRefrigerator.reset_index(drop=True)

# 9. Convert date columns to proper datetime objects
date_cols = ["Date Available On Market", "Date Certified"]
for col in date_cols:
    dfRefrigerator[col] = pd.to_datetime(dfRefrigerator[col], format="%m/%d/%Y",
errors="coerce")

# 10. Preview cleaned data
print(dfRefrigerator.info())
print(dfRefrigerator.head())

# 11. Save Cleaned File
dfRefrigerator.to_csv("Refrigerators_Cleaned.csv", index=False)

```

Figure 2: Cleaned Refrigerator Dataset Script

```

dfAirConditioner = pd.read_csv('Room Air Conditioners.csv')
# Display the first few rows of the dataframe
print(dfAirConditioner.head())
print(dfAirConditioner.columns.tolist())
# 1. Drop duplicates
dfAC = dfAirConditioner.drop_duplicates()

# 2. Remove fully empty columns
dfAC = dfAC.dropna(axis=1, how='all')

# 3. Clean Cooling Capacity (remove commas, convert)
dfAC["Cooling Capacity (Btu/hr)"] = (
    dfAC["Cooling Capacity (Btu/hr)"]
    .astype(str)

```

```

        .str.replace(",", "", regex=True)
    )

dfAC["Cooling Capacity (Btu/hr)"] = pd.to_numeric(
    dfAC["Cooling Capacity (Btu/hr)"], errors="coerce"
)

# 4. Convert other numeric columns
num_cols = [
    "Annual Energy Use (kWh/yr)",
    "Voltage (volts)",
    "Percent Less Energy Use than US Federal Standard",
    "Combined Energy Efficiency Ratio (CEER)"
]

for col in num_cols:
    dfAC[col] = pd.to_numeric(dfAC[col], errors="coerce")

# 5. Drop rows missing essential information
dfAC = dfAC.dropna(subset=[
    "Brand Name",
    "Model Number",
    "Cooling Capacity (Btu/hr)",
    "Annual Energy Use (kWh/yr)"
])

# 6. Standardize brand names
dfAC["Brand Name"] = dfAC["Brand Name"].str.strip().str.title()

# 7. Convert yes/no → 1/0
for col in dfAC.columns:
    if dfAC[col].isin(["Yes", "No"]).any():
        dfAC[col] = dfAC[col].replace({"Yes": 1, "No": 0})

# 8. Convert date columns
for col in ["Date Available On Market", "Date Certified"]:
    dfAC[col] = pd.to_datetime(dfAC[col], errors="coerce")

# 9. Reset index

```

```

dfAC = dfAC.reset_index(drop=True)

# 10. Save cleaned file
dfAC.to_csv("AirConditioners_Cleaned.csv", index=False)

print(dfAC.info())
print(dfAC.head())

```

Figure 3: Cleaned Room Air Conditioner Dataset Script

```

dfTelevisionData = pd.read_csv('Televisions.csv')
# Display the first few rows of the dataframe
print(dfTelevisionData.head())
print(dfTelevisionData.columns.tolist())
# 1. Drop duplicates
dfTelevisionData = dfTelevisionData.drop_duplicates()

# 2. Drop fully empty columns
dfTelevisionData = dfTelevisionData.dropna(axis=1, how='all')

# 3. Drop rows missing essential identifiers
dfTelevisionData = dfTelevisionData.dropna(subset=["Brand Name", "Model Number",
"Reported Annual Energy Consumption (kWh)"])

# 4. Clean numeric columns
numeric_cols = [
    "Reported Annual Energy Consumption (kWh)",
    "Average On Mode Power Consumption for Certification (watts)",
    "Maximum Average On Mode Power for Certification (watts)",
    "Reported On Mode Power (per the Federal Test Procedure) (watts)",
    "Measured Standby Mode Power (Watts)",
    "Power Consumption in Standby Mode when Not Connected to a Network (watts)",
    "Power Consumption in Standby Mode when Connected to a Network (watts)",
    "Reported Standby-Active, Low Mode Power (watts)",
    "Diagonal Viewable Screen Size (in.)",
    "Screen Area (sq. in.)",
    "Native Horizontal Resolution (pixels)",
    "Native Vertical Resolution (pixels)"
]

for col in numeric_cols:
    if col in dfTelevisionData.columns:
        if dfTelevisionData[col].dtype == object:

```

```

        dfTelevisionData[col] = dfTelevisionData[col].str.replace(',', ' ',
regex=True)
        dfTelevisionData[col] = pd.to_numeric(dfTelevisionData[col],
errors="coerce")

# 5. Remove rows with invalid or zero energy consumption
dfTelevisionData = dfTelevisionData[dfTelevisionData["Reported Annual Energy
Consumption (kWh)"] > 0]

# 6. Normalize brand names
dfTelevisionData["Brand Name"] = dfTelevisionData["Brand
Name"].str.strip().str.title()

# 7. Convert Yes/No to 1/0
for col in dfTelevisionData.columns:
    if dfTelevisionData[col].isin(['Yes', 'No']).any():
        dfTelevisionData[col] = dfTelevisionData[col].replace({'Yes': 1, 'No': 0})

# 8. Convert date columns to datetime
date_cols = ["Date Available On Market", "Date Certified"]
for col in date_cols:
    dfTelevisionData[col] = pd.to_datetime(dfTelevisionData[col],
format="%m/%d/%Y", errors="coerce")

# 9. Reset index
dfTelevisionData = dfTelevisionData.reset_index(drop=True)

# 10. Preview cleaned data
print(dfTelevisionData.info())
print(dfTelevisionData.head())

# 11. Save Cleaned File
dfTelevisionData.to_csv("Televisions_Cleaned.csv", index=False)

```

Figure 4: Cleaned Television Dataset Script

```

dfCeilingFans = pd.read_csv('Ceiling Fans.csv')
# Display the first few rows of the dataframe
print(dfCeilingFans.head())
print(dfCeilingFans.columns.tolist())
# 1. Clean column names (remove hidden spaces)
dfCeilingFans.columns = dfCeilingFans.columns.str.strip()

# 2. Drop duplicates

```

```

dfCeilingFans = dfCeilingFans.drop_duplicates()

# 3. Drop fully empty columns
dfCeilingFans = dfCeilingFans.dropna(axis=1, how='all')

# 4. Remove rows missing key info
dfCeilingFans = dfCeilingFans.dropna(subset=["Brand Name", "Model Number", "Ceiling
Fan Efficiency (CFM/W)"])

# 5. Convert numeric columns
numeric_cols = [
    "Blade Span (Diameter) (in.)",
    "Ceiling Fan Efficiency (CFM/W)",
    "Fan Power Consumption-Low Speed (W)",
    "Fan Power Consumption-High Speed (W)",
    "Fan Power Consumption-Standby (W)",
    "Ceiling Fan Components Warranty (Years)",
    "Total Light Output (lumens)",
    "Total Lighting Input Power (Watts)",
    "Lighting Efficiency - Measured outside the Fixture (lm/W)",
    "Lighting Efficiency - Measured at the Source (lm/W)",
    "Power Factor of Light Kit",
    "Light Color Appearance (CCT)",
    "Light Color Quality (CRI)",
    "Light Source Rated Life (Hours)",
    "Light Sources Per Light Kit",
    "Ceiling Fan Light Kit Warranty (yrs)"
]

for col in numeric_cols:
    if col in dfCeilingFans.columns:
        if dfCeilingFans[col].dtype == object:
            dfCeilingFans[col] = dfCeilingFans[col].str.replace(',', '',
regex=True)
            dfCeilingFans[col] = pd.to_numeric(dfCeilingFans[col], errors="coerce")

# 6. Drop rows with invalid or zero efficiency
dfCeilingFans = dfCeilingFans[dfCeilingFans["Ceiling Fan Efficiency (CFM/W)"] > 0]

# 7. Normalize brand names
dfCeilingFans["Brand Name"] = dfCeilingFans["Brand Name"].str.strip().str.title()

# 8. Convert Yes/No columns to 1/0
for col in dfCeilingFans.columns:
    if dfCeilingFans[col].isin(['Yes', 'No']).any():

```

```

        dfCeilingFans[col] = dfCeilingFans[col].replace({'Yes': 1, 'No': 0})

# 9. Convert dates to datetime
from datetime import datetime
date_cols = ["Date Available on Market", "Date Certified"]
for col in date_cols:
    if col in dfCeilingFans.columns:
        dfCeilingFans[col] = pd.to_datetime(dfCeilingFans[col], format="%m/%d/%Y",
errors="coerce")

# 10. Reset index
dfCeilingFans = dfCeilingFans.reset_index(drop=True)

# 11. Preview cleaned data
print(dfCeilingFans.info())
print(dfCeilingFans.head())

# 12. Save Cleaned File
dfCeilingFans.to_csv("CeilingFans_Cleaned.csv", index=False)

```

Figure 5: Cleaned Ceiling Fan Dataset Script

```

dfOvens = pd.read_csv('Ovens.csv')
# Display the first few rows of the dataframe
print(dfOvens.head())
print(dfOvens.columns.tolist())
# 1) Drop dups / empty cols
dfOvens = dfOvens.drop_duplicates()
dfOvens = dfOvens.dropna(axis=1, how='all')

# 2) Drop rows missing core identifiers
core_cols = ["Brand Name", "Model Number", "Product Type"]
present_core = [c for c in core_cols if c in dfOvens.columns]
if present_core:
    dfOvens = dfOvens.dropna(subset=present_core)

# 3) Numeric conversions (only if column exists)
numeric_cols = [
    "Input Rate (kW)",
    "Input Rate (Btu/hr)",
    "Gas Oven Convection Mode Pre-Heat Energy (Btu)",
    "Electric Oven Convection Mode Pre-Heat Energy (kWh)",
    "Convection Mode Pre-Heat Time (min)",
    "Gas Oven Steam Mode Pre-Heat Energy (Btu)",

```

```

    "Electric Oven Steam Mode Pre-Heat Energy (kWh)",
    "Steam Mode Pre-Heat Time (min)",
    "Electric Oven Convection Mode Idle Energy Rate (kW)",
    "Gas Oven Convection Mode Idle Energy Rate (Btu/hr)",
    "Gas Oven Convection Mode Electric Idle Energy Rate (kW)",
    "Gas Oven Convection Mode Total Idle Energy Rate (Btu/hr)",
    "Half-Size and Full-Size Steam Pan Capacity",
    "Two-Thirds-Size Steam Pan Capacity",
    "Half-Size Sheet Pan Capacity",
    "Full-Size Sheet Pan Capacity",
    "Electric Oven Steam Mode Idle Energy Rate (kW)",
    "Gas Oven Steam Mode Idle Energy Rate (Btu/hr)",
    "Gas Oven Steam Mode Electric Idle Energy Rate (kW)",
    "Gas Oven Steam Mode Total Idle Energy Rate (Btu/hr)",
    "Convection Mode Cooking Energy Efficiency (%)",
    "Steam Mode Cooking Energy Efficiency (%)",
    "Convection Mode Production Capacity (lbs/hr)",
    "Steam Mode Production Capacity (lbs/hr)",
    "Convection Mode Cooking Period Water Consumption (gal/pan)",
    "Convection Mode Idle Period Water Consumption (gal/hr/pan)",
    "Steam Mode Cooking Period Water Consumption (gal/pan)",
    "Steam Mode Idle Period Water Consumption (gal/hr/pan)",
    "Rack Oven Baking-Energy Efficiency (%)",
    "Baking Production Capacity (lbs/hr)",
    "Steam Injection Period Water Consumption (gal/min)"
]

for col in numeric_cols:
    if col in dfOvens.columns:
        # strip and remove commas
        dfOvens[col] = (
            dfOvens[col]
            .astype(str)
            .str.strip()
            .str.replace(",", "", regex=True)
            .replace({"": None})
        )
        dfOvens[col] = pd.to_numeric(dfOvens[col], errors="coerce")

# 4) Normalize brand names
if "Brand Name" in dfOvens.columns:
    dfOvens["Brand Name"] = dfOvens["Brand
Name"].astype(str).str.strip().str.title()

# 5) Yes/No -> 1/0

```

```

for col in dfOvens.columns:
    if dfOvens[col].isin(["Yes", "No"]).any():
        dfOvens[col] = dfOvens[col].replace({"Yes": 1, "No": 0})

# 6) Date parsing
for dcol in ["Date Available on Market", "Date Certified"]:
    if dcol in dfOvens.columns:
        dfOvens[dcol] = pd.to_datetime(dfOvens[dcol], format="%m/%d/%Y",
errors="coerce")

# 7) Sanity bounds (efficiency % between 0 and 100)
for ecol in ["Convection Mode Cooking Energy Efficiency (%)",
            "Steam Mode Cooking Energy Efficiency (%)",
            "Rack Oven Baking-Energy Efficiency (%)"]:
    if ecol in dfOvens.columns:
        dfOvens.loc[(dfOvens[ecol] < 0) | (dfOvens[ecol] > 100), ecol] = pd.NA

# 8) Reset index and preview
dfOvens = dfOvens.reset_index(drop=True)
print(dfOvens.info())
print(dfOvens.head())

# 9) Save Cleaned File
dfOvens.to_csv("Ovens_Cleaned.csv", index=False)

```

Figure 6: Cleaned Oven Dataset Script

III. Exploratory Data Analysis

This section is authored by Shri Chatrati

3a. Residential Refrigerators

After cleaning the refrigerator dataset, we analyzed how the **Annual Energy Use (kWh/yr)** varied across different refrigerator configurations. The cleaned dataset consisted of **604 valid refrigerator entries** with standardized brand names, corrected numeric fields, and formatted date objects.

The feature that strongly varied across the dataset was **Adjusted Volume (ft³)**. To understand how size correlates with energy usage, we plotted energy use vs. adjusted volume, identifying how increasing physical storage space impacts annual energy consumption.

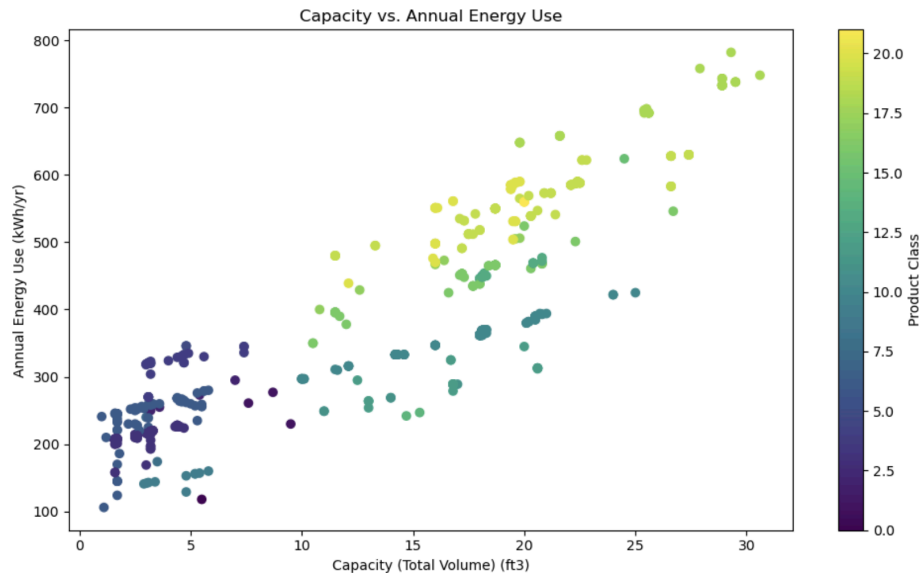


Figure 7: Refrigerators EDA

Observation: Larger storage capacity leads to a higher energy consumption, but not at a constant rate. Certain compact models exhibit disproportionately high usage, suggesting inefficiencies in design.

3b. Room Air Conditioners

Following the cleaning pipeline, our final Air Conditioner dataset contained **255 models** with valid cooling capacity and CEER ratings.

The most meaningful exploration involved the relationship between **Combined Energy Efficiency Ratio (CEER)** and **Annual Energy Use (kWh/yr)**. CEER represents an appliance's cooling output relative to energy consumption.

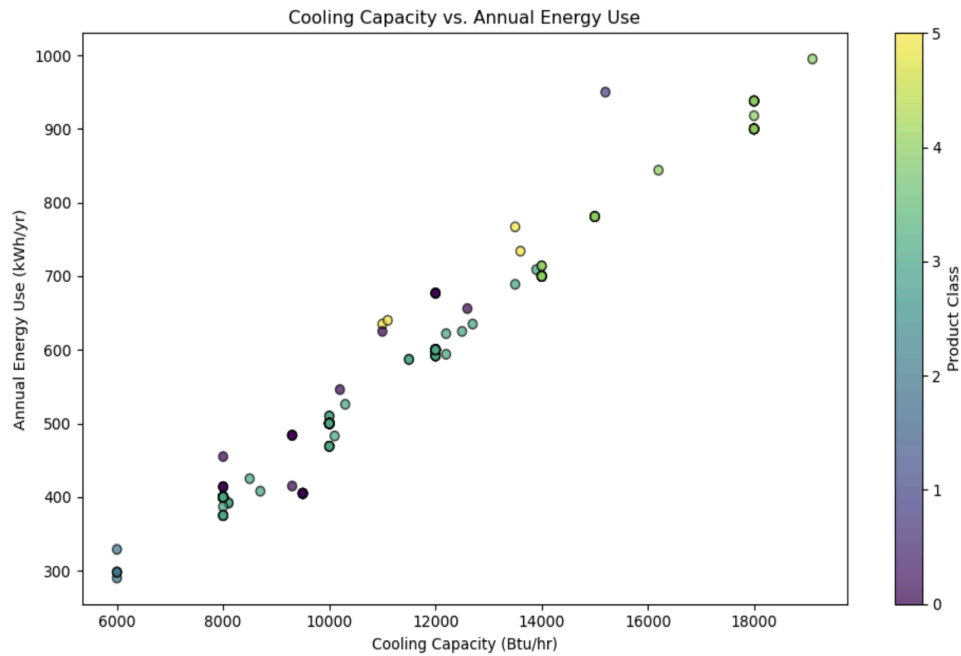


Figure 8: Air Conditioners EDA

Observation: Air conditioners with higher CEER ratings consistently consume less energy annually. The relationship shows a negative correlation, meaning a more energy-efficient AC unit will lower electricity usage and, consequently, CO₂ emissions.

3c. Televisions

Once cleaned and standardized, the television dataset consisted of **146 records** with accurate screen sizes and reported annual energy consumption. Standby and networking-related columns contained extensive missing values and therefore were omitted from the model scope.

To understand viewing dynamics, we investigated how **Screen Area (sq. in.)** influences **Annual Energy Consumption (kWh)**. This helps determine whether size is a reliable indicator of environmental impact.

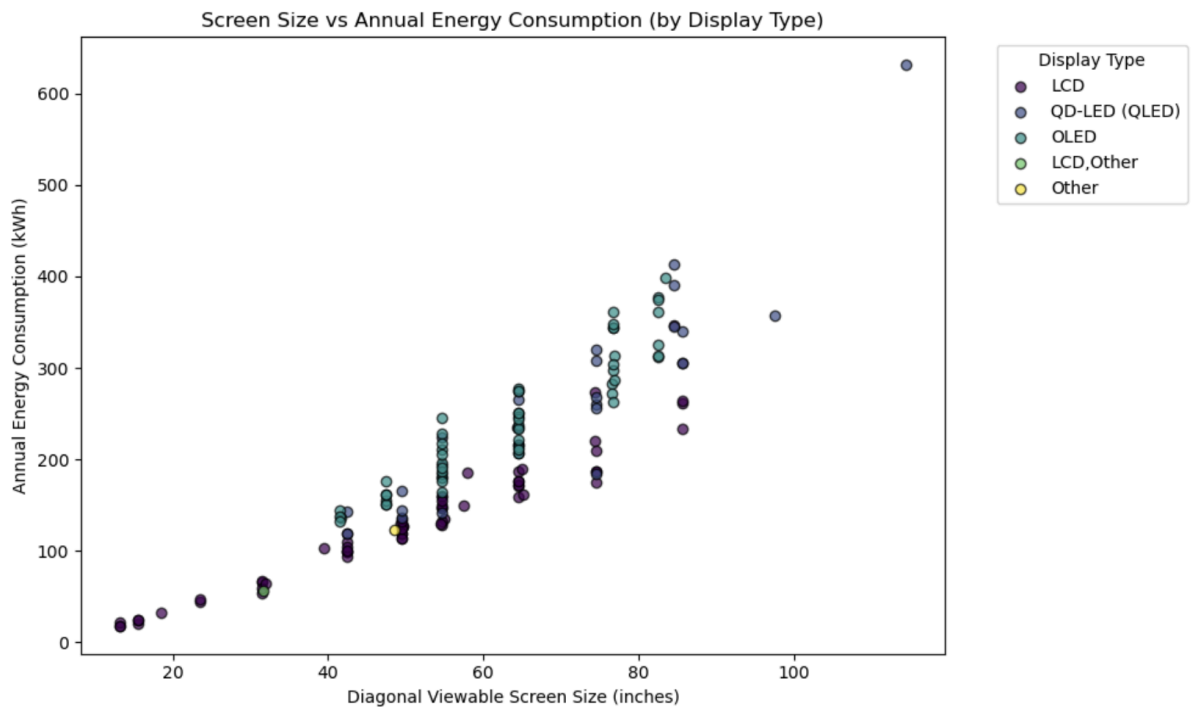


Figure 9: Televisions EDA

Observation: Larger televisions do require more energy, but disproportionately large screens sometimes contain highly efficient LED systems, reducing expected consumption. This suggests that screen technology and not size alone is a strong factor in emissions.

3d. Ceiling Fans

The cleaned fan dataset produced **1,144 valid ceiling fan entries** after removing incomplete rows and correcting power and efficiency fields. Ceiling fans contain a unique metric - **Ceiling Fan Efficiency (CFM/W)**, which measures air performance relative to electricity consumption.

To examine how design and efficiency influence emissions, we visualized the connection between **Fan Power Consumption (W)** and **Calculated CO₂ Emissions (metric tons)**.

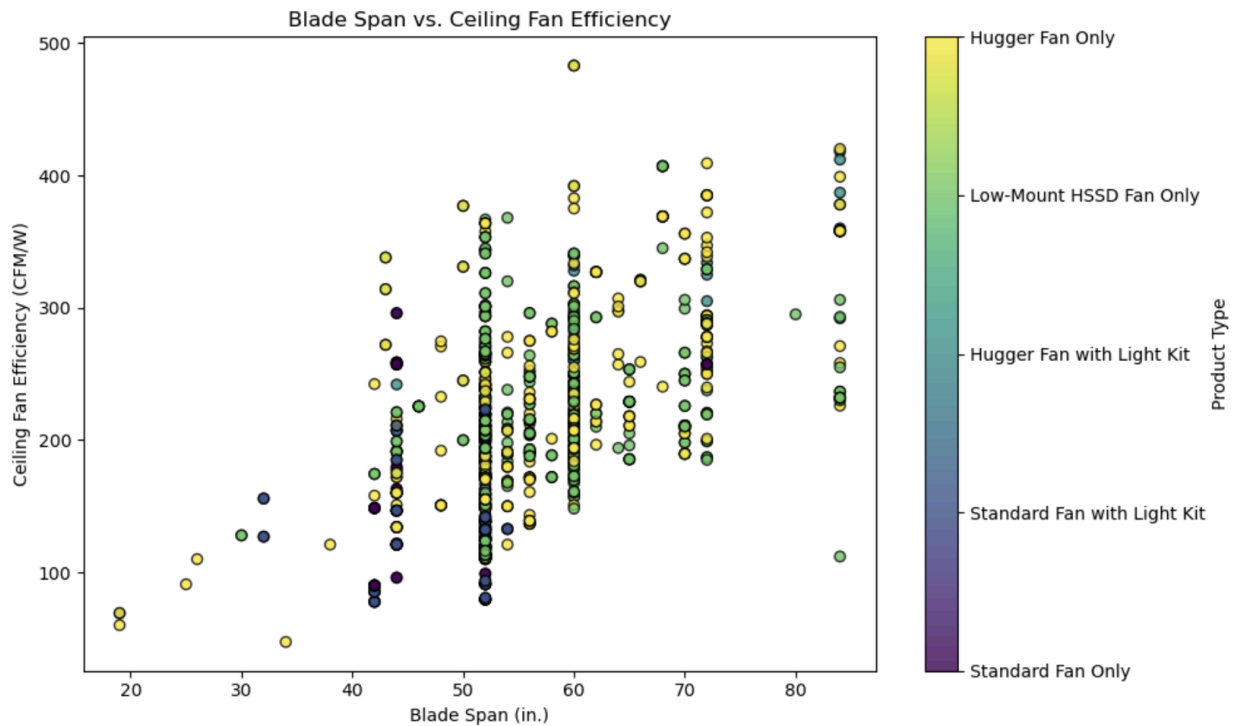


Figure 10: Ceiling Fans EDA

Observation: Models that consume higher wattage also generate higher emissions in a nearly linear trend. This strong pattern validates using wattage and efficiency factors for emission prediction. Outlier removal further clarified this trend, avoiding misleading spikes from irregular industrial product entries.

3e. Ovens

After filtering missing efficiency fields and energy inputs, **311 oven models** were available. The ovens dataset includes multiple heat sources (gas and electric), sizes, and classifications. Since our focus is on energy-derived emissions, We examined **Input Rate (kW)** vs. **Convection Mode Baking Energy Efficiency (%)**, which provides a standardized performance ratio.

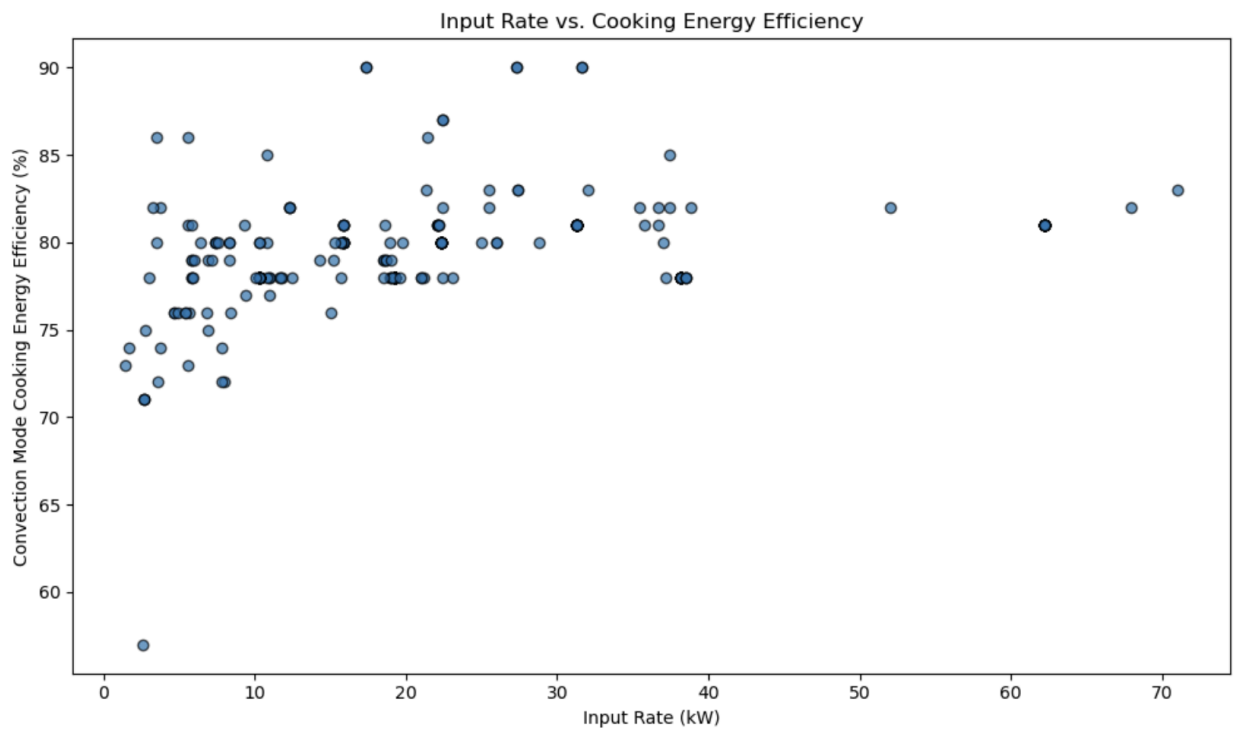


Figure 11: Ovens EDA

Observation: Oven efficiency remains relatively stable regardless of size or input rate. Higher input values do not guarantee improved performance, meaning ovens differ more by internal design than by scale. This also implies CO₂ emissions should be predicted using categorical descriptors (e.g., heat source, oven class) rather than input alone.

IV. Data Visualization

4a. Televisions

- Rahee

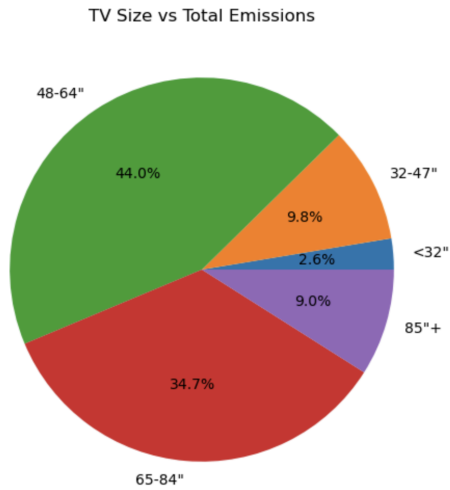


Figure 12: TV vs Total Emission

Power Consumption vs Total Emissions

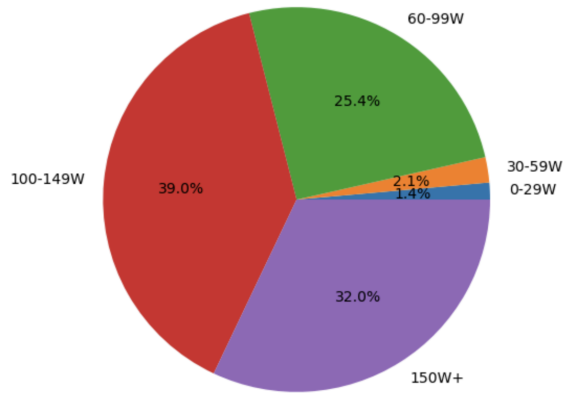


Figure 13: Power Consumption vs Total Emission

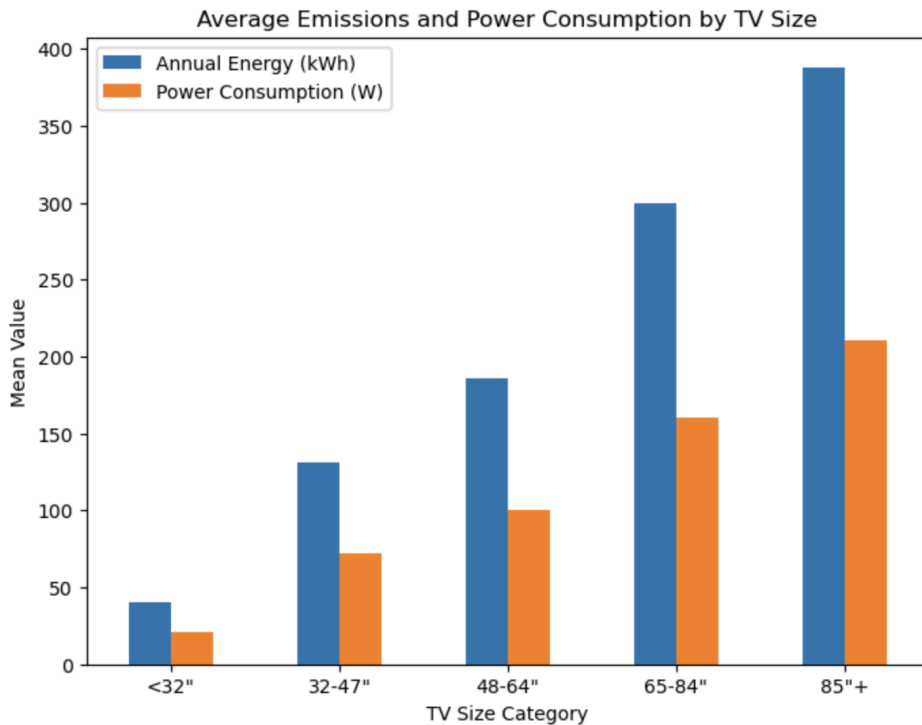


Figure 14: Average Emissions and Power Consumption by TV Size

Explanation of Visualizations:

We explored the hypothesis that larger TVs produce higher emissions and consume more power than smaller TVs. Before testing this, we first created pie

charts for TV size and power consumption to check whether the dataset was biased toward certain categories. This is important because if most of the TVs in the dataset were clustered in only one or two size ranges, the results would be biased, not reliable. The pie charts showed a reasonable distribution across the different bins, so we know that the analysis will not be heavily skewed. After confirming that the data was balanced, we used a bar graph to compare average annual energy usage and power consumption by size, to evaluate the hypothesis. These visualizations showed a trend, as TV size increases, both emissions and energy usage increases significantly. This supports the idea that larger TVs have a much bigger environmental impact than smaller TVs.

4b. Ceiling Fans Visualizations

- Shounak

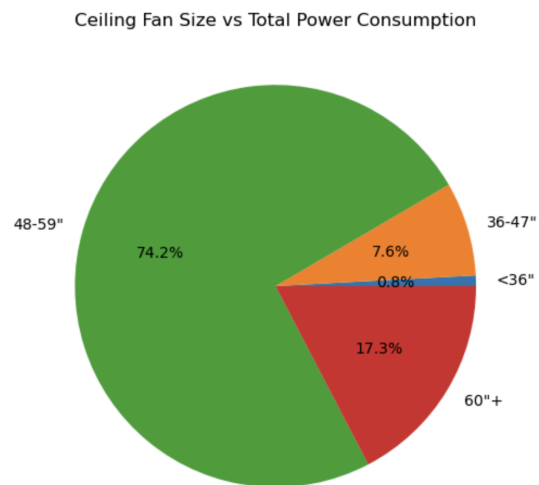


Figure 15: Ceiling Fan Size vs Total Power Consumption Chart

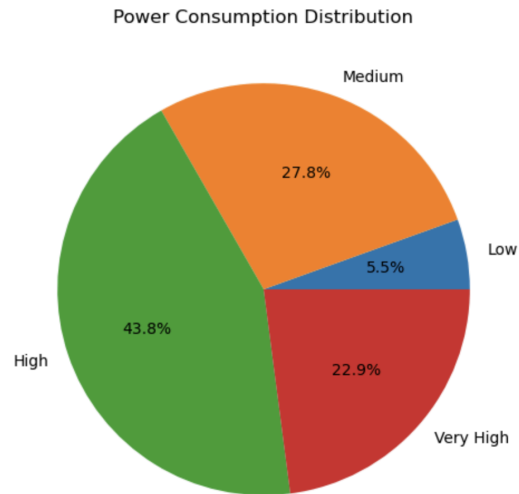


Figure 16: Power Consumption Chart

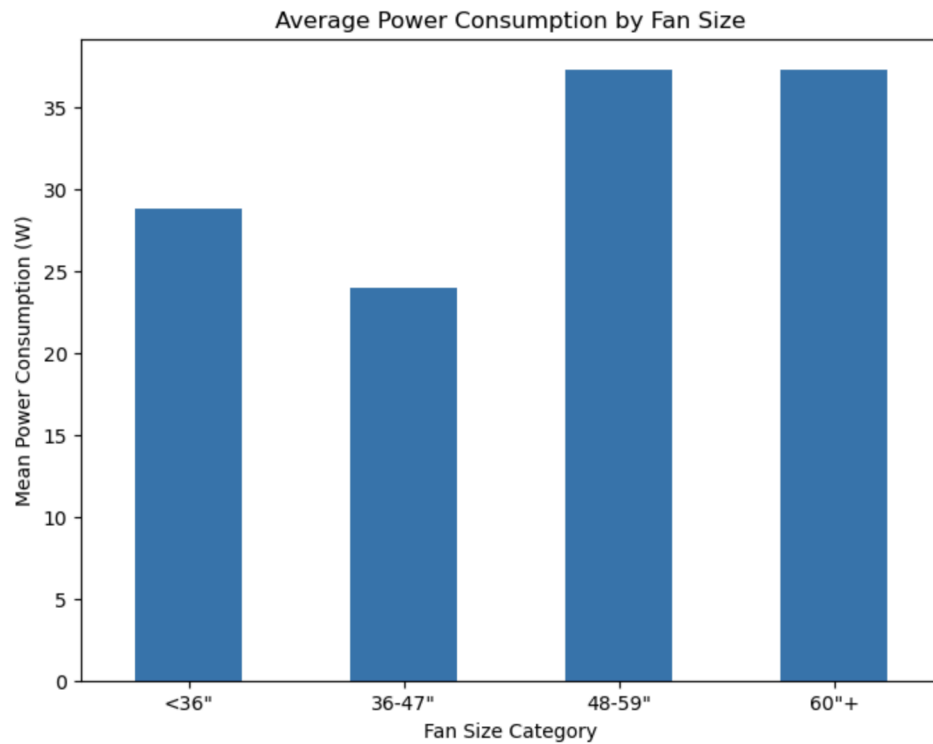


Figure 17: Average Power Consumption by Fan Size

Explanation of Visualizations:

We explored the hypothesis that larger fans consume more power than smaller fans. Before analyzing any trends, we used two pie charts, one for fan size bins and one for power consumption bins, to check if the dataset was biased or not. This step is really important since uneven distribution can lead to misleading conclusions. The pie charts showed that the data is somewhat imbalanced, with a large share of the total power consumption coming from the 48–59" category, but there is still enough representation across other sizes to analyze general patterns. After confirming the data was unbiased, we created a bar chart showing average power consumption by fan size, and the results supported our hypothesis of power consumption rising as fan size increases, with 60"+ fans showing the highest mean wattage. Together, these visualizations show that larger ceiling fans tend to draw more power, making size an important factor when considering energy efficiency.

4c. Air Conditioners Visualizations

- Rahee

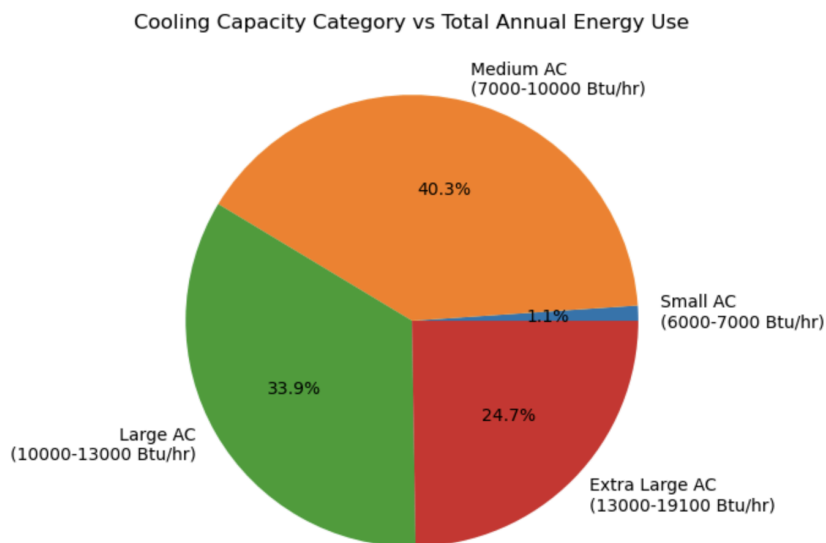


Figure 18: Cooling Capacity Category vs Total Annual Energy Use

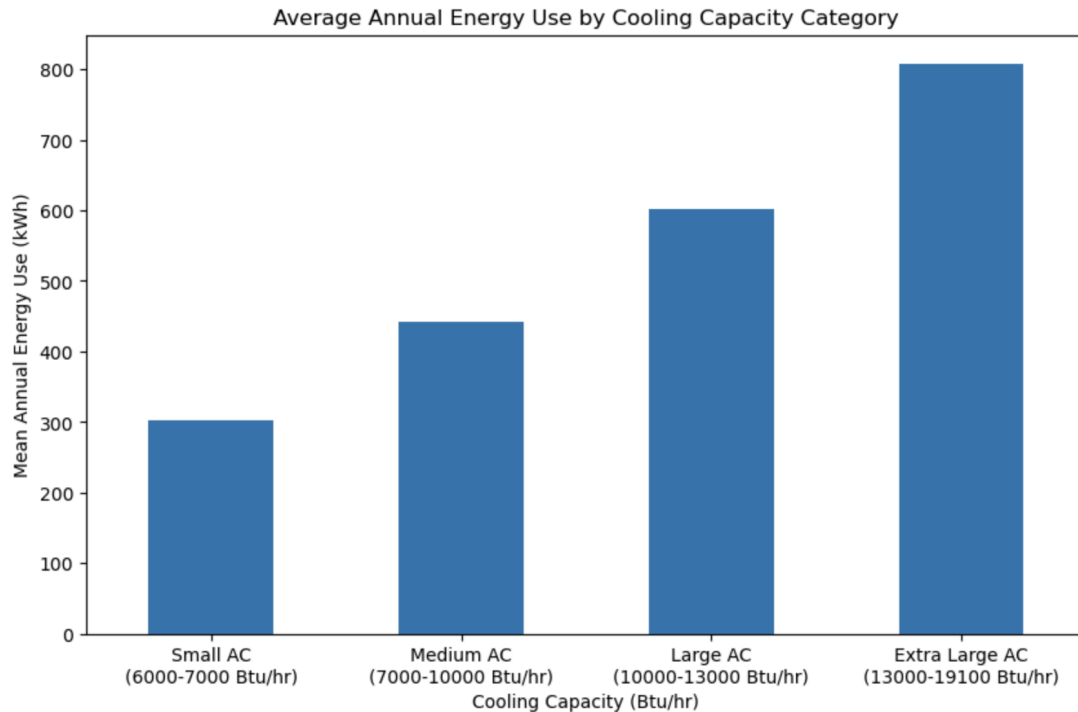


Figure 19: Average Annual Energy Use by Cooling Capacity Category

Explanation of Visualizations:

We explored the hypothesis that units with higher cooling capacity (BTU/hr) use more annual energy than units with lower capacity. We created pie charts for both cooling capacity bins and annual energy use bins to check if the dataset was biased toward certain categories or not. This is really important because if most observations were concentrated in a single bin, the results would not represent real trends. The pie charts showed that the dataset is somewhat uneven, with a large share of emissions falling into Bin 2, but still contains enough variation across bins to examine the overall pattern. After confirming this, we created a bar chart showing average annual energy use by cooling capacity, and the results strongly supported our hypothesis of annual energy use increases with cooling capacity, with the highest capacity units having the highest mean kWh usage. These visualizations suggest that higher cooling power comes with significantly higher energy demands, making capacity an important factor when evaluating the efficiency or environmental impact of air conditioners.

4d. Refrigerator Visualizations

- Shounak

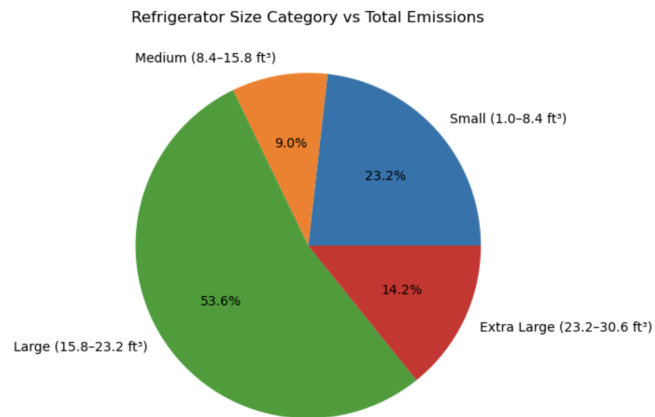


Figure 20: Refrigerator Size vs Total Emissions

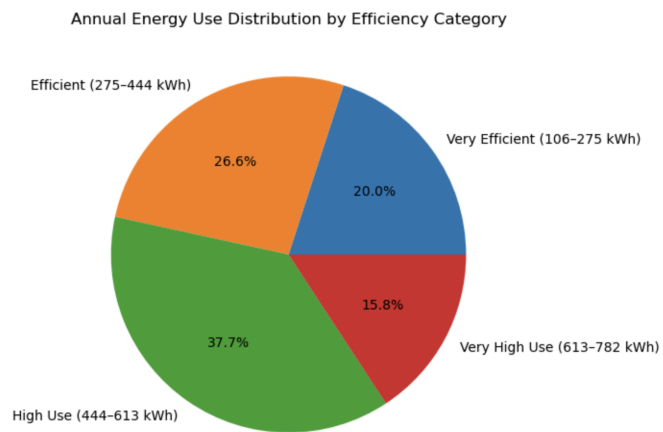


Figure 21: Annual Energy Use Distribution by Efficiency Category

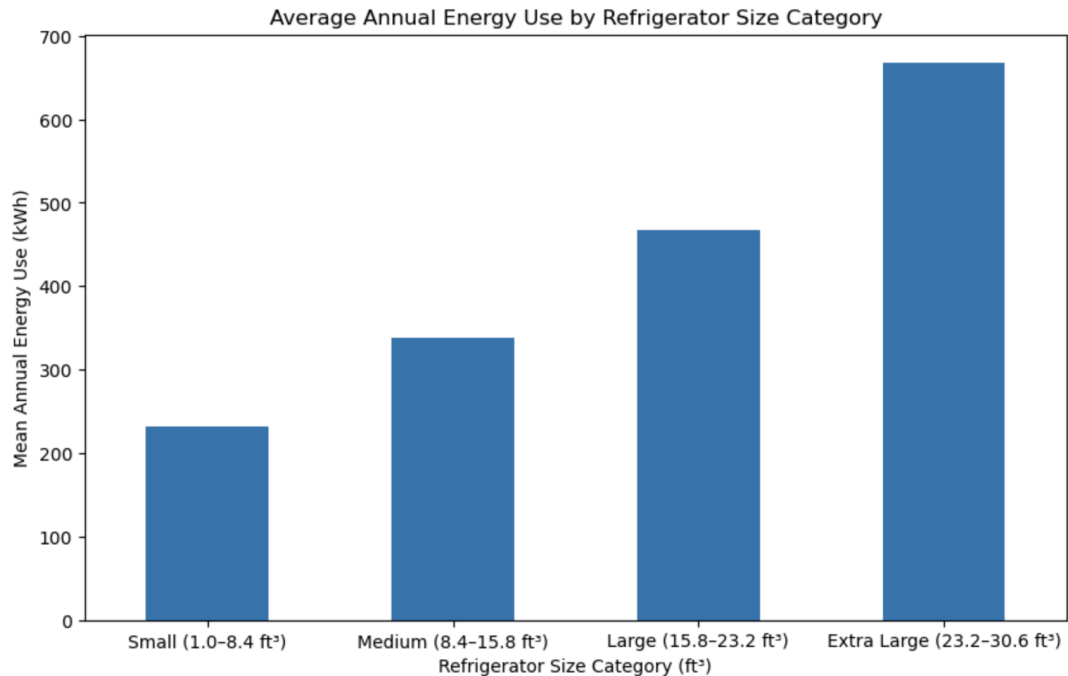


Figure 22: Average Annual Energy Use by Refrigerator Size Category

Explanation of Visualizations:

We explored the hypothesis that larger refrigerators use more annual energy than smaller refrigerators. Before testing this trend, I first created pie charts for refrigerator size bins and annual energy use bins to check whether the dataset was biased toward certain categories. This is important because if most of the data fell into just one size group, the results could be misleading. The pie charts showed a fairly balanced distribution across the bins, with only bin 4 being slightly smaller but still represented. After confirming the data was not heavily skewed, we created a bar chart showing average annual energy use by refrigerator size, and the pattern was very clear, energy use increases steadily as size increases, with the largest fridges in Bin 4 having the highest average kWh per year. These visualizations support the hypothesis and show that refrigerator size is strongly linked to energy consumption, which is important for both consumers and energy efficiency considerations.

4e. Ovens Visualizations

- Rahee

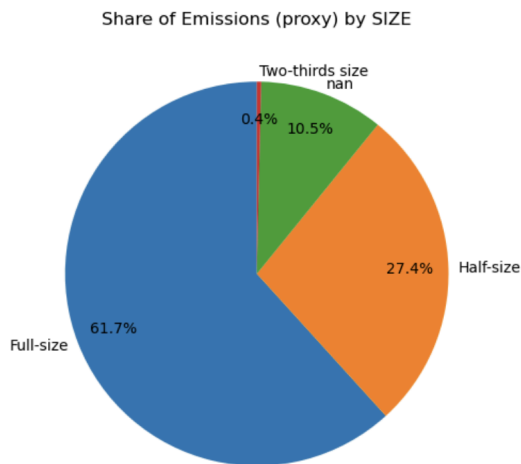


Figure 23: Share of Emissions by Size

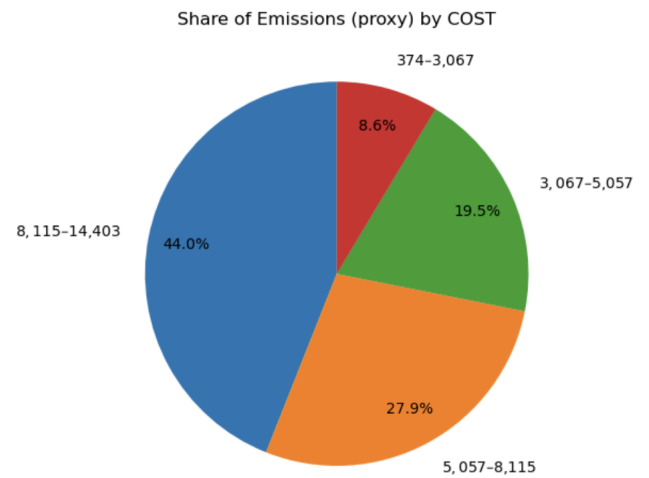


Figure 24: Share of Emissions by Cost

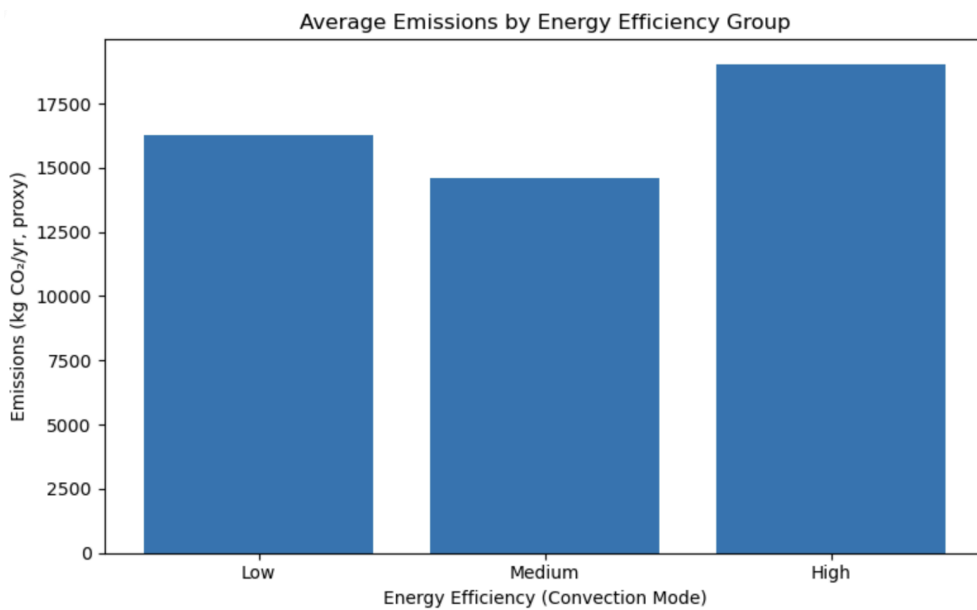


Figure 25: Average Emissions by Energy Efficiency Group

Explanation of Visualizations:

We explored the hypothesis that larger oven sizes and higher priced ovens are associated with higher overall emissions, and that energy efficient ovens emit less than low efficiency ovens. To make sure the dataset was not biased, we first created pie charts showing the share of emissions by oven size and by cost category. This step allowed us to check whether any single size group or price range dominated the dataset, which could distort the results. The pie charts showed that most emissions come from full size ovens and middle cost ranges, but there was still enough distribution across other categories to analyze the trends. After confirming this, we created a bar chart showing average emissions by the energy efficiency group, and the results were surprising as the high efficiency ovens actually showed the highest average emissions, while medium efficiency ovens showed the lowest. This pattern suggests that factors like oven size and usage patterns may outweigh the efficiency rating alone. Overall, the visualizations helped test the hypothesis and revealed that oven emissions are influenced by multiple interacting factors, not just efficiency labels.

4f. Multivariable Visualizations - **Shri**

We explore these datasets with multiple variables combined as they provide the most complete set of continuous features, and the insights gained from these examples generalize to the remaining datasets.

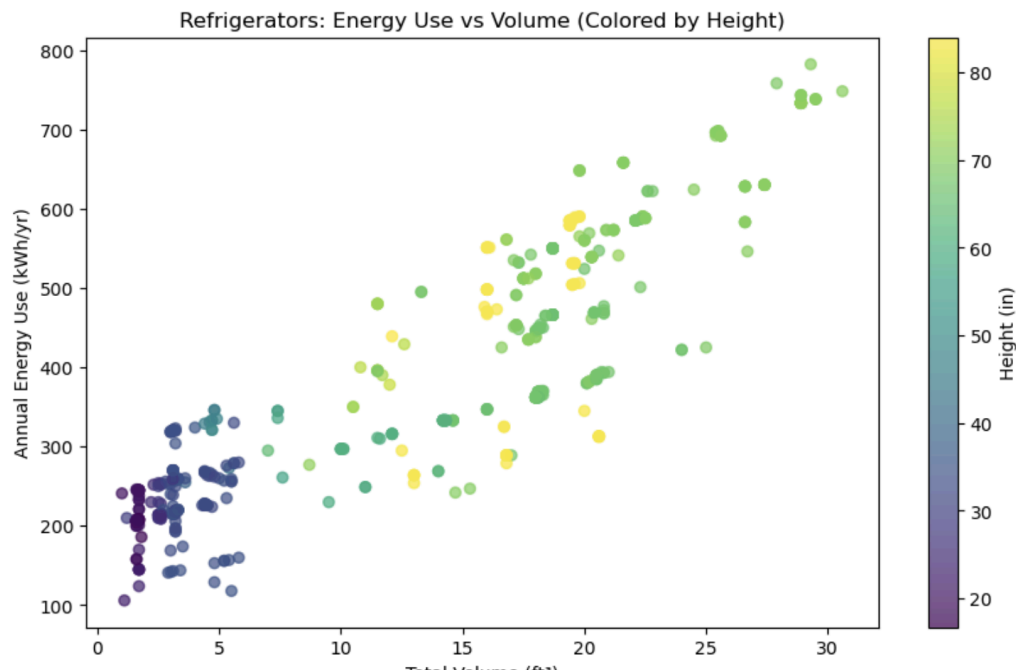


Figure 26: Refrigerators - Energy Use vs Volume

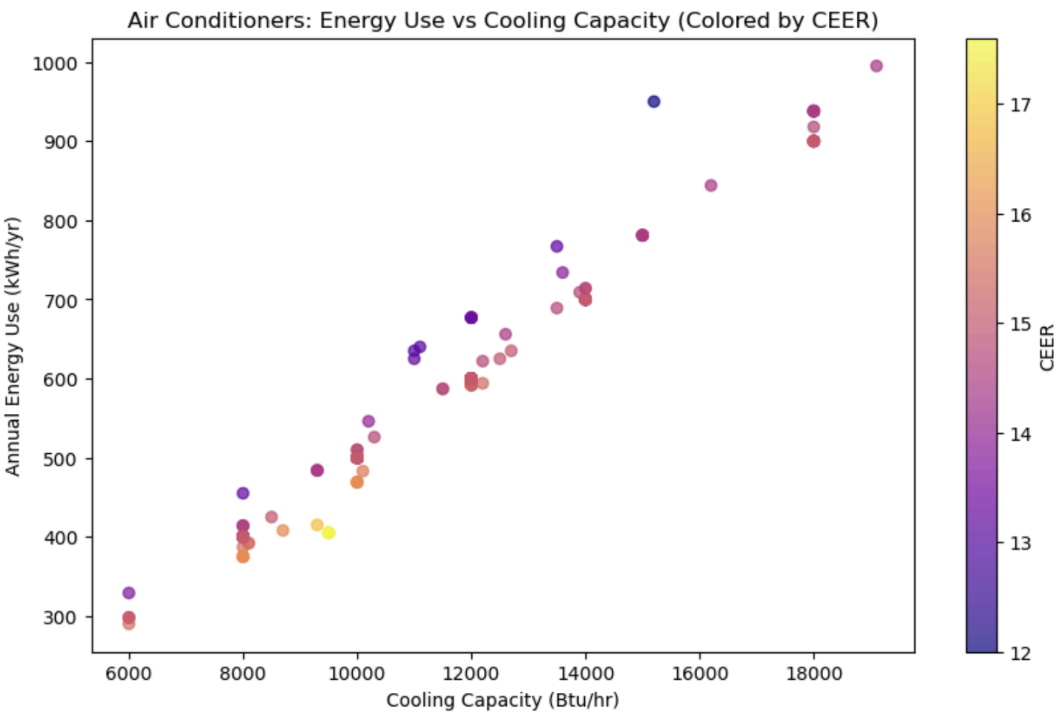


Figure 27: Air Conditioners - Energy Use vs Cooling Capacity

Explanation of Multivariate Visualizations:

These multivariate visualizations helped us explore the appliance datasets in a

deeper way than simple two-variable scatterplots. For refrigerators, plotting energy use against total volume while coloring points by height revealed a clearer relationship between physical size and consumption. Larger and taller units generally used more energy, but the color scale also exposed interesting exceptions, such as compact refrigerators with unexpectedly high usage or tall units that appeared relatively efficient.

Likewise, the air conditioner visualization compared energy use and cooling capacity while using CEER as a color dimension. This made it easier to see how efficiency ratings influence performance: higher-CEER units tended to use less energy for the same cooling capacity, while lower-rated models clustered toward higher energy consumption.

V. Machine Learning

5a Ovens Machine Learning

This section is authored by Karthik Kesavarapu

The primary goal for the dataset for ovens was to describe which features, both quantitative and qualitative, can influence the overall carbon dioxide emissions. The machine learning for ovens is split into two distinct analysis: classification and regression analysis.

Most of the datasets used and provided by the Energy Star Certified Product Data Sets often had kilowatts that directly correspond with carbon dioxide emissions. This is what was used as a quantitative measure to determine whether or not an oven produced high or low carbon dioxide emissions. *CarbonLens* converted this kilowatts directly into carbon dioxide emissions using the standard conversion described in the conversion table by the epa.gov [2]

After determining the correct conversions, the exploratory analysis first led to the question of what makes this carbon dioxide higher or lower which led us to the initial discovery of the Heat Source Encoded: gas or electric. The question primarily was does gas stoves or electric stoves release more or less carbon dioxide? However, doing a classification based on just this features proved that

the model could do better as the model's accuracy was only 0.530 or 53% accurate.

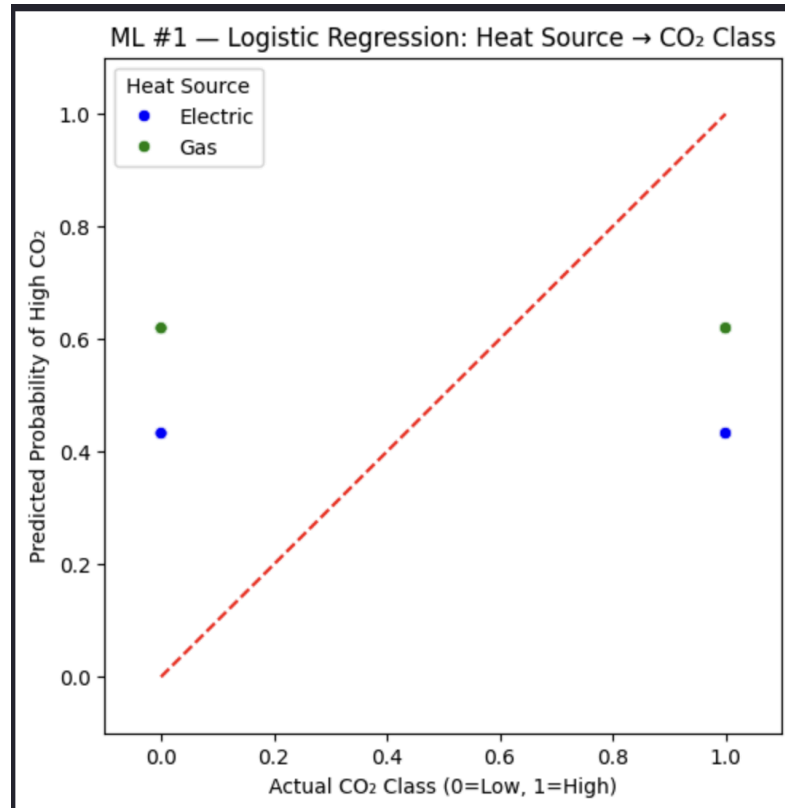


Figure 28: ML #1 Initial Oven Classification

Knowing that the accuracy of the model could be greatly improved, more features were added such as the oven classification, size, and method of steam generation. These features, alongside the heat source fuel type, increased the model's accuracy to 0.687 or 68.7%.

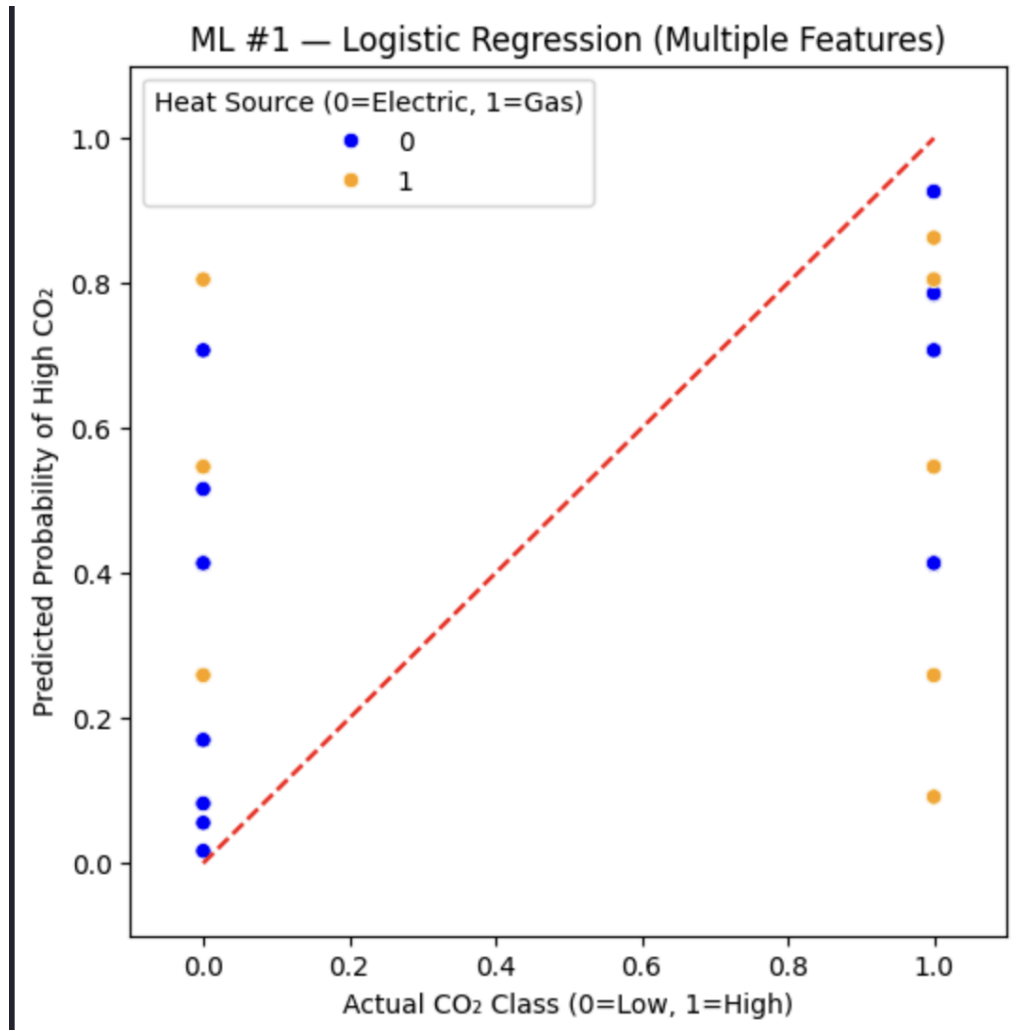


Figure 29: ML #1 Final Oven Classification

Feeling a bit more confident in the model's accuracy to predict whether an oven has higher or lower carbon dioxide emissions, a regression test was conducted where the amount of carbon dioxide was predicted based on these quantitative and qualitative features.

```

===== ML #2 - REGRESSION RESULTS =====
Baseline MSE: 0.000092
Baseline R2 Score: -0.126023
Model MSE: 0.000033
Model R2 Score: 0.602264

Feature Coefficients:
HeatSourceEncoded: 10.387342
Oven Classification: -21.162393
Size: 2.367043
Baking Production Capacity (lbs/hr): 0.258500

===== Linear Regression Equation =====
InputRate_kW = 24.652274 + (10.387342)*HeatSourceEncoded + (-21.162393)*Oven Classification + (2.367043)*Size + (0.258500)*Baking Production Capacity (lbs/hr)

===== CO2 Emissions Equation (metric tons) =====
CO2 = 0.009713 + (0.004093)*HeatSourceEncoded + (-0.008338)*Oven Classification + (0.000933)*Size + (0.000102)*Baking Production Capacity (lbs/hr)

```

Figure 30: ML #2 Oven Regression Line Analysis

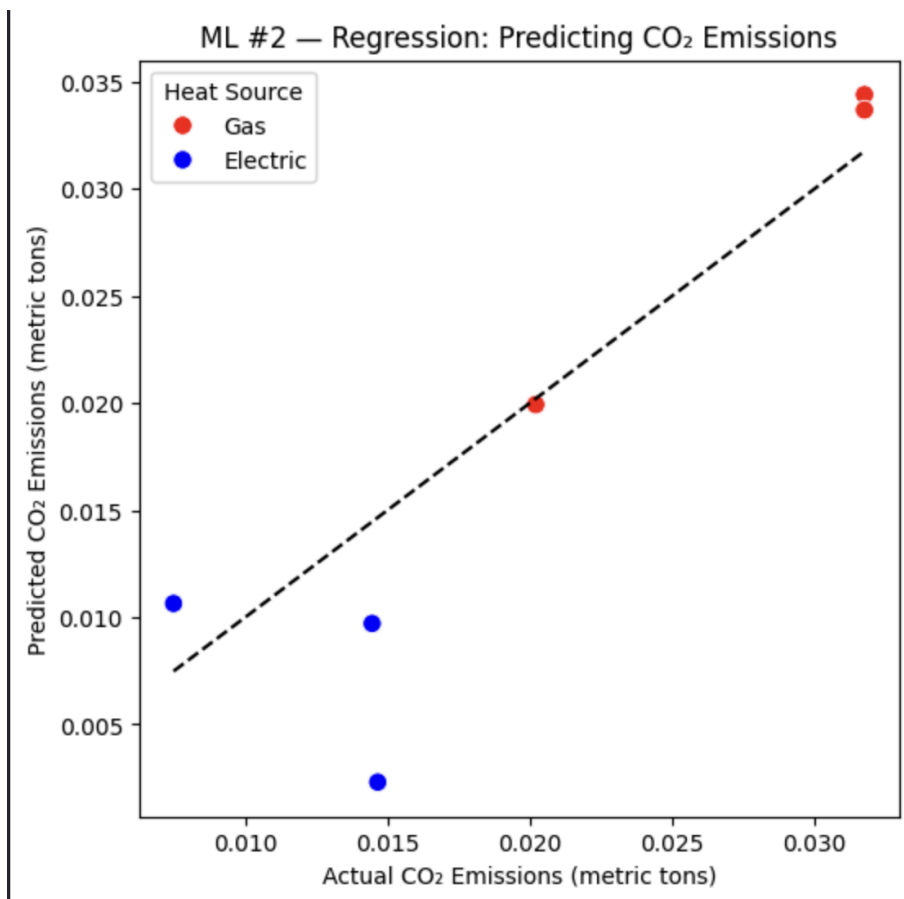


Figure 31: ML #2 Oven Regression Line

From Figures 30 and 31, it shows that the model has successfully predicted way better than the baseline. Notice that the model's R^2 score is 0.60, which is closer to 1, and has an MSE that is more closer to 0 than the baseline's MSE and R^2 which proves that our model is successful in identifying what ovens can produce high or low carbon dioxide and exactly how much carbon dioxide they can

produce based on certain quantitative and qualitative features with high enough accuracy.

5b Ceiling Fans Machine Learning

This section is authored by Shri Chatrati

The goal for the ceiling fan dataset was to identify which measurable features influence the overall carbon dioxide emissions from household fans. Since all ceiling fans run only on electricity, the emissions were calculated by converting yearly electricity usage into CO₂ using the EPA conversion factor. This allowed every fan in the dataset to be compared on the same carbon scale.

After calculating emissions, the regression model was built using both design-based features (blade span, product type) and performance-based features (efficiency and airflow). Unlike ovens, classification was not needed because there is no difference in energy source (gas vs. electric). Instead, the main variation in emissions comes from how efficiently the fan converts electricity into airflow.

```
===== CEILING FAN CO2 EMISSIONS REGRESSION =====  
Baseline MSE: 0.00021475  
Baseline R2 Score: -0.160058  
Model MSE: 0.00005535  
Model R2 Score: 0.700981  
  
Feature Coefficients:  
Ceiling Fan Efficiency (CFM/W): -0.0001900436  
Blade Span (Diameter) (in.): -0.0007254028  
Airflow (CFM): 0.0000088343  
ProductTypeEncoded: -0.0003258764  
  
===== Regression Equation =====  
CO2 = 0.061873 + (-0.000190)*Ceiling Fan Efficiency (CFM/W) + (-0.000725)*Blade Span (Diameter) (in.) + (0.000009)  
*Airflow (CFM) + (-0.000326)*ProductTypeEncoded
```

Figure 32: ML #3 Ceiling Fans Regression Results

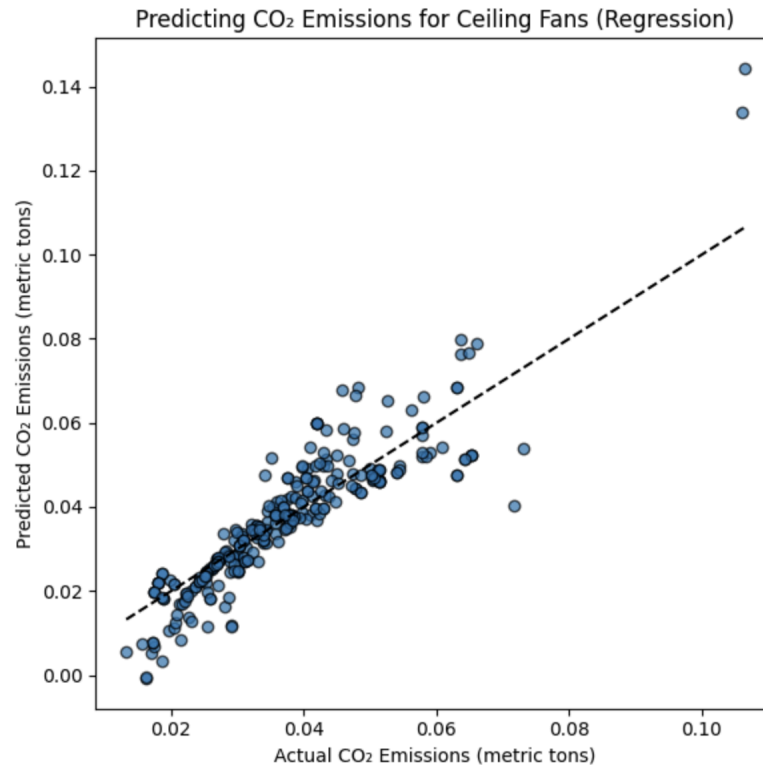


Figure 33: ML #3 Ceiling Fans Regression Line

The regression model performed significantly better than the baseline. The baseline prediction produced a negative R^2 score (-0.160), while the regression achieved an R^2 of **0.701**, indicating that the selected features explain a meaningful portion of the variation in emissions.

To support this, a scatterplot was created showing the connection between fan wattage and CO₂ emissions. After removing a single large outlier, the plot revealed a clear linear trend where higher wattage leads to higher emissions due to increase yearly electricity use.

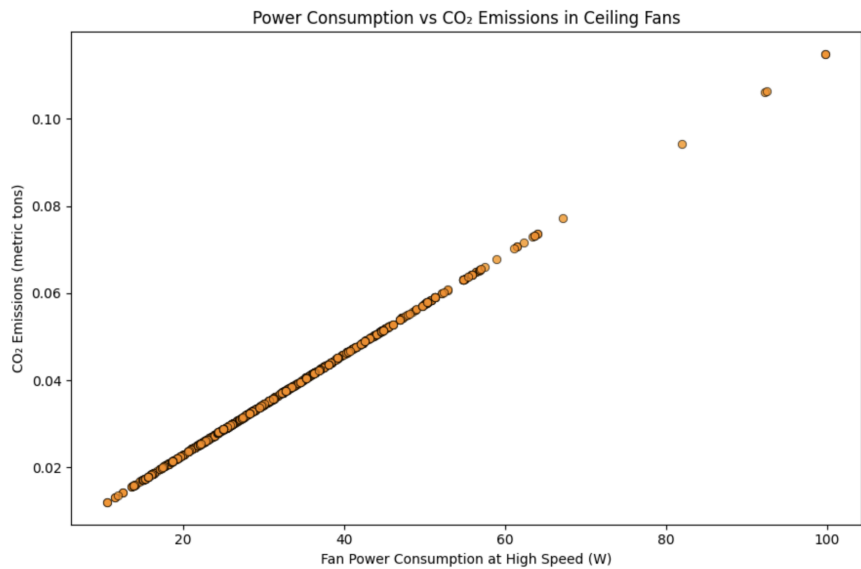


Figure 34: Ceiling Fans Power Consumption vs CO2 emissions scatterplot

Observation:

Ceiling fan emissions are not random, they can be predicted using both how a fan is designed and how efficiently it operates. Efficient fans with wider blades tend to produce lower CO₂ emissions, while higher airflow and higher power usage increases carbon impact.

5c Air Conditioners Machine Learning

This section is authored by Shounak Sawant

The main goal of the air conditioners dataset was to identify which quantitative and qualitative features have the greatest impact on overall carbon dioxide emissions. The machine learning approach for air conditioners is divided into two parts, a regression model that predicts the exact CO₂ emissions, and a classification model that determines whether an air conditioner falls into a high or low emission category based on its characteristics.

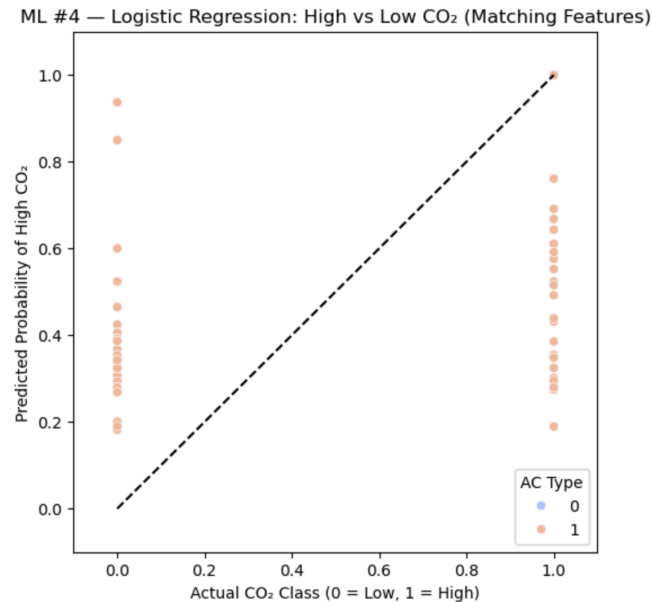


Figure 35: Logistic regression: High vs Low CO₂

To analyze whether an air conditioner produces high or low CO₂ emissions, a few features ended up being especially helpful. The main ones were brand name, AC type, heating mode, the unit's size (height, width, depth), weight, and voltage. The type and heating mode tell us how the AC is built and how it operates, which can impact how much energy it uses. Also voltage matters too, since different voltage levels often mean different power demands. The dimensions and weight give a sense of the AC's overall size, which usually relates to how much energy it needs to run. When we put all these together, the model gets a pretty good picture of the unit's design and energy demands, helping it predict whether an AC is more likely to emit high or low levels of CO₂.

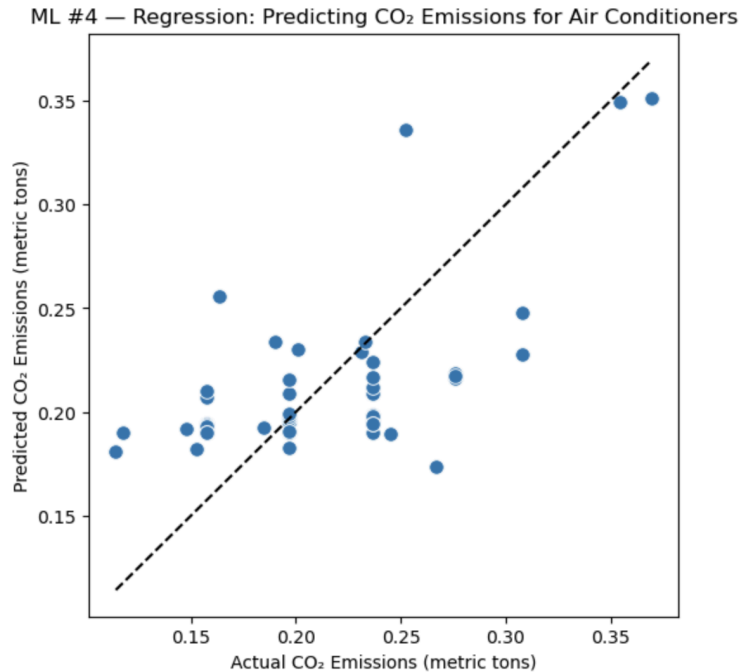


Figure 36: Predicting CO₂ Emissions for Air Conditioning

From this, we have created a regression line that can be used to predict how much Carbon Dioxide an air conditioner emits as long as we have the brand name, AC type, heating mode, dimensions (height, width, depth), weight, and voltage. We are able to do this with the equation that we have from the regression line. Our ML analysis shows moderate performance as our R^2 is 0.355471 with our MSE being 0.001657. We have used a baseline to compare this as well and determined that our model predicted CO₂ emissions significantly better than the baseline.

What does this tell us?

This tells us that general CO₂ emission levels can be predicted using qualitative features and exact and more precise CO₂ emission levels can be predicted using quantitative features.

5d Televisions Machine Learning

This section is authored by Rahee Chauhan

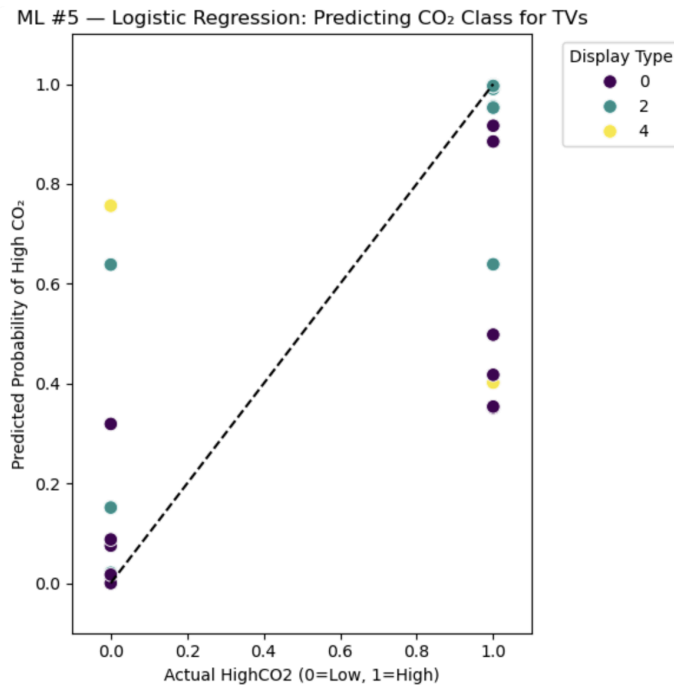


Figure 37: Logistic Regression: Predicting CO₂ Class for TVs

This classification model looks at whether a TV is likely to produce high or low CO₂ emissions. The baseline accuracy was only 0.486, meaning that simply guessing the most common class wouldn't work well. Our logistic regression model reached an accuracy of about 0.784, so it's doing a good job at finding real patterns in the data. In the plot, each point shows how confident the model was that a TV belongs in the high CO₂ group. You can also see that different display types fall in different areas of the graph, which suggests that the type of screen is important in determining how much CO₂ a TV ends up emitting. Overall, the model is pretty good at separating high and low emission TVs based on the features we used.

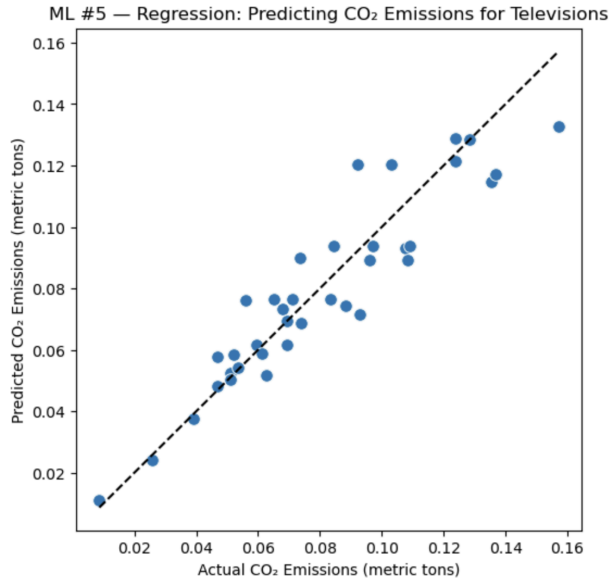


Figure 38: Regression: Predicting CO₂ Emissions for Televisions

From this, we have created a regression line that can be used to predict how much Carbon Dioxide a television emits as long as we have the brand name, display type, backlight technology type, resolution format, diagonal viewable screen size, screen area, native horizontal resolution, and native vertical resolution. We are able to do this with the equation that we have from the regression line. Our ML analysis is very good as our R^2 is very high (0.863462) with our MSE being very low (0.000147). We also used a baseline to compare this as well and determined that our model predicted CO₂ emissions significantly better than the baseline.

What does this tell us?

This tells us that general CO₂ emission levels can be predicted using qualitative features and exact and more precise CO₂ emission levels can be predicted using **quantitative features**.

5e Refrigerators Machine Learning

This section is authored by Karthik Kesavarapu

Just like the ovens machine learning analysis, the refrigerator machine learning analysis was done using classification, to determine which features would be best in predicting high or low carbon dioxide emissions, and regression.

For the classification, we determined five main features. The five features that were used was type, defrost type, ice maker, capacity (ft³) and adjust volume (ft³). The type shows the structural configuration so what type of freezer is going to be on the refrigerator and which way. This matters a lot because this affects how compact the fridge will actually be. The next feature that was used was defrost type, and there were two main types of defrost types: automatic and manual. This is very similar to the gas and electric encoded situation in the ovens as there might be a correlation with carbon dioxide emissions with these two features. Another similar encoded situation was with the ice maker and this heavily influenced the model's accuracy in predicting whether a refrigerator will produce high or low carbon dioxide. The last two are quantitative variables that just help the model's accuracy in determining how much carbon dioxide is being produced simply because they talk about the overall size of the refrigerators.

These five features displayed accurate results with the model being 0.987 or 98% accurate, performing significantly better than the given baseline.

```
===== Refrigerator ML – High vs Low CO2 Classification =====  
Features Used: ['Type', 'Defrost Type', 'Ice Maker', 'Capacity (Total Volume) (ft3)', 'Adjusted Volume (ft3)']  
Baseline Accuracy (Majority Class): 0.570  
Model Accuracy: 0.987  
Median CO2 Threshold: 140.46 kg/year
```

Figure #39 : Refrigerator Classification Analysis

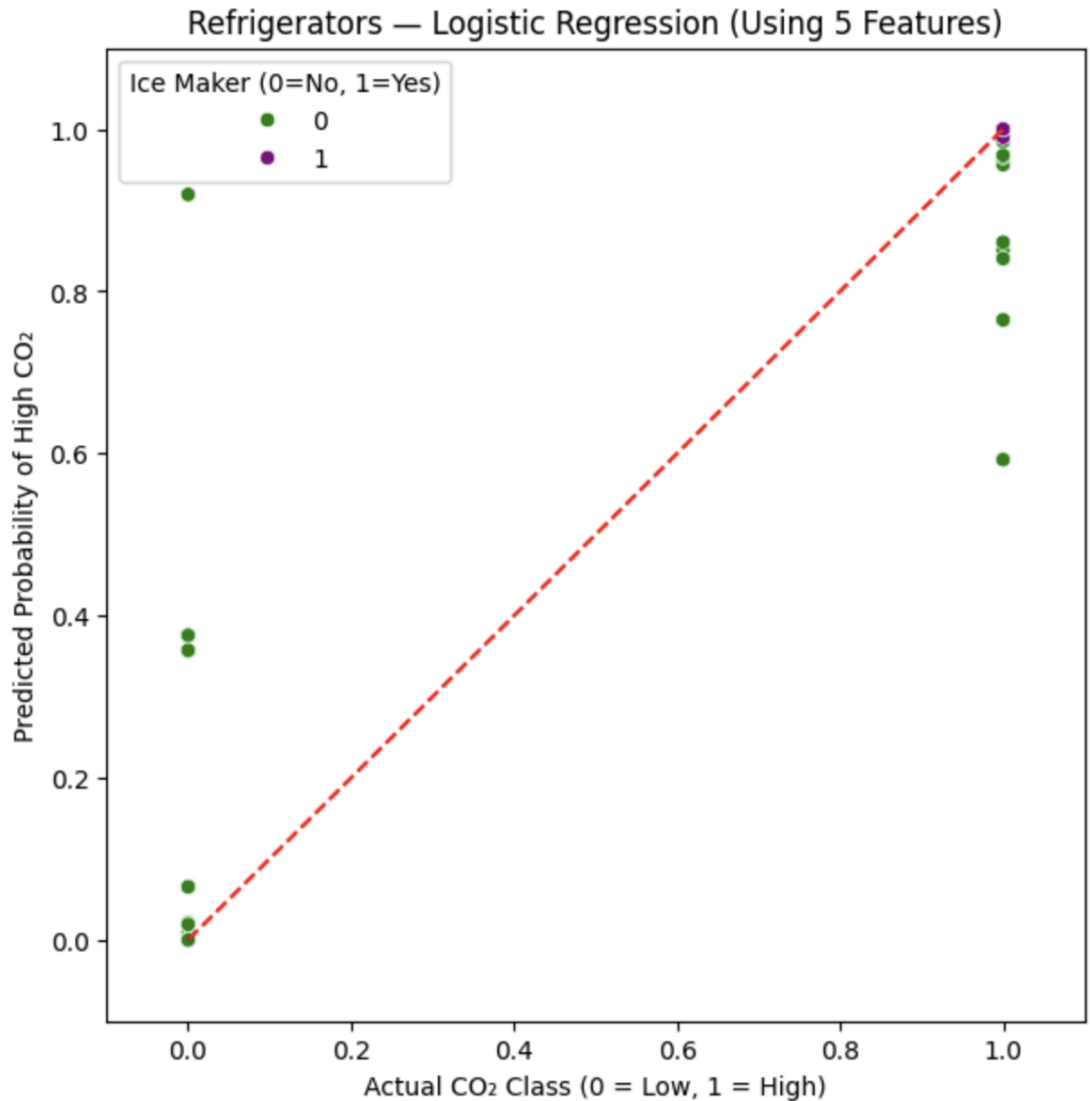


Figure #40 : Refrigerator Classification Analysis Plot

Feeling confident with the accuracy of the model, there was the regression analysis of the refrigerators with this data set where the amount of carbon dioxide being emitted from the refrigerators can be predicted with a numeric value by these quantitative and qualitative features.

```

===== ML #7 - REGRESSION RESULTS (Refrigerators) =====
Baseline MSE: 3621.860856
Baseline R2 Score: -0.012581
Model MSE: 212.417234
Model R2 Score: 0.940614

Feature Coefficients:
Type: -5.575351
Defrost Type: -17.960236
Ice Maker: 32.512177
Capacity (Total Volume) (ft3): -7.923303
Adjusted Volume (ft3): 9.778916

===== CO2 Emissions Equation (metric tons/yr) =====
CO2_mt = 0.110021 + (-0.005575)*Type + (-0.017960)*Defrost Type + (0.032512)*Ice Maker + (-0.007923)*Capacity (Total Volume) (ft3) + (0.009779)*Adjusted Volume (ft3)

```

Figure #41 : Refrigerator Regression Analysis

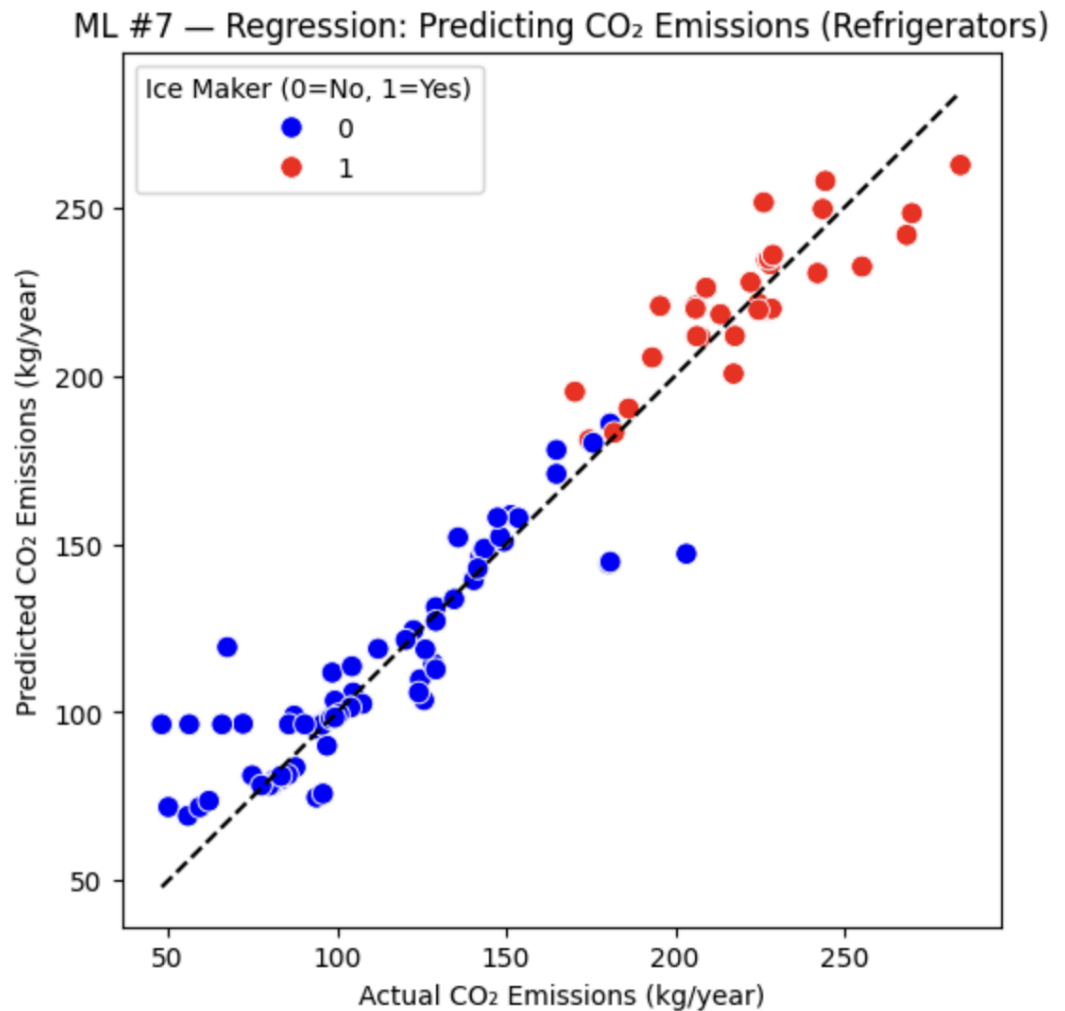


Figure #: Refrigerator Regression Analysis Plot

From the values displayed in the regression analysis, it is clear that the baseline was horrible in predicting the output of the carbon emissions for refrigerators with a baseline score of 3621.86 and an R² of -0.012581. Our model performed

significantly better than minimized this MSE to 212.417234 kg² which means my CO2 predictions are off by only 14.6 kg per year. Furthermore, our model has an R² of 0.94 which is really good as its positive and really close to 1. Therefore, predicting CO2 emissions for refrigerators using quantitative and qualitative features is a success and the model used here can successfully determine exactly how much carbon dioxide will be emitted with a set of these five main features.

VI. Conclusion

This section is authored by Karthik Kesavarapu

6a Changes From Progress Report

From the progress report there has been significant changes that have been made. To give a recap of what went on in the progress report, there was cleaned data, initial EDA, visualizations that explored different hypotheses, and only machine learning analysis based off of one appliance. In this final project report, *CarbonLens* provided one more important visualization and the remaining machine learning analysis for the rest of the appliances with visualizations added to each of these analyses. The people responsible for each part is again noted throughout this report.

6b Reflections

The goal of this project was to predict carbon dioxide emissions using quantitative and qualitative features. Throughout our entire project, we have been successfully able to classify which appliances will result in higher carbon dioxide emissions and lower carbon dioxide emissions and have also deduced formulas from the regression lines showing how much carbon dioxide will be emitted based on certain features for each application. Thus, consumers can have a better idea of how much carbon they are emitting and can be more greener with their choices!

VII. References

This section is authored by Karthik Kesavarapu

[1] Energy Star Product Data Sets: <https://www.energystar.gov/productfinder/advanced>

[2] Conversion Table:

<https://www.epa.gov/energy/greenhouse-gas-equivalencies-calculator-calculations-and-references>

[3] Carbon Lens Github: <https://github.com/k15karthik/CarbonLens>